

Month 2 Week 2 Assignment Report

Objectives

- Understand the architecture and components of Convolutional Neural Networks (CNNs).
- Learn about convolutional and pooling layers and their role in feature extraction.
- Apply the knowledge to practical computer vision tasks using real-world datasets.
- Implement and utilize callbacks to improve model training.

Resources

- [Deep Learning for Computer Vision](#)
- [Understanding Convolutional Layer and Pooling Layers](#)

Task 1: Understanding CNNs

Q1. What are the advantages of convolutional layers over fully connected layers in image processing tasks?

Answer:

A convolutional layer used in CNNs is fundamentally different from a fully connected layer used in some standard ANNs and offers following key advantages.

1. Parameter Sharing: In a convolutional layer, a single filter (or kernel), which is a small matrix of weights, is used to scan across the entire image. This means the same set of weights (parameters) is used to detect the same feature (e.g., a vertical edge, a curve, or a patch of color) no matter where it appears in the image. In contrast, a fully connected layer would require a separate weight for every single pixel for every neuron, leading to an explosion in the number of parameters. Parameter sharing makes the network far more efficient and reduces the risk of overfitting.
2. Sparsity of Connections: Neurons in a convolutional layer are only connected to a small, local region of the input image which is known as the receptive field. This is based on the idea that in an image, nearby pixels are related, but a pixel in the top-left corner has very little to do with a pixel in the bottom-right. Fully connected layers, on the other hand, connect every neuron to every pixel in the input image,

ignoring the spatial structure and which will lead to creating unnecessary connections.

3. Translation Invariance: A direct benefit of parameter sharing is that if the network learns to detect a feature in one part of the image, it can automatically detect it in any other part. For example, if it learns to recognize an eye in the top-left corner, it can use that same knowledge to find an eye in the center or bottom-right. This makes the model robust to the position of objects in an image.

Q2. How does pooling help in reducing the computational complexity of a CNN?

Answer:

Pooling layers reduce computational complexity primarily by performing down-sampling. They reduce the spatial dimensions (the height and width) of the feature maps that come out of the convolutional layers.

Here's how it works:

1. A pooling layer takes a feature map and a window size (e.g., 2x2 pixels).
2. It slides this window across the feature map.
3. For each window, it aggregates the features within it into a single value (e.g., by taking the maximum value or the average).

By doing this, a 16x16 feature map might become an 8x8 feature map. When this smaller map is passed to the next layer, there are far fewer parameters to train and fewer calculations to perform. This makes the model faster, more memory-efficient, and also helps in making the feature detection more robust by summarizing features in a local region.

Q3. Compare different types of pooling layers (max pooling, average pooling). What are their respective advantages and disadvantages?

Answer:

Max pooling and average pooling are the two most common types of pooling. Following are some pros and cons of each.

Max Pooling:

It takes the maximum value from the feature map covered by the window.

Advantages:

- It is very effective at capturing the most prominent or "activated" features in a region. For example, if the goal is to detect sharp edges, corners, or bright spots, max pooling will preserve these strong signals.
- It tends to work better in practice for most computer vision tasks because it focuses on the key, distinguishing features and ignores weaker activations.

Disadvantages:

- It can be too aggressive by discarding all other information in the pooling window. If other, less intense features are also important, they are completely lost.

Average Pooling

It calculates the average of all values in the feature map covered by the window.

Advantages:

- It provides a smoother, more generalized summary of the features in a region, as it takes all activations into account.
- It can be useful when the overall context of a feature is more important than its sharpest part, such as for background textures or smooth gradients.

Disadvantages:

- It can dilute the most important feature by averaging it with other, less important ones. A strong activation signal might be "watered down," making it harder for the network to identify key features.
- In practice, it is often found to be less effective than max pooling for classification tasks.

Task 2: Implementing a CNN

1. Build a Simple ANN and CNN on the MNIST Dataset([MNIST](#)):

1. Architectures of ANN & CNN

The following models were constructed to classify digits from the MNIST dataset.

Simple ANN Architecture:

- Layers:
 - Flatten: This input layer transformed the 2D image arrays (28x28) into a 1D vector of 784 pixels.
 - Dense: A fully connected hidden layer with 128 neurons using the ReLU activation function.
 - Dense (Output): The final layer with 10 neurons (for digits 0-9) using the Softmax activation function to output probabilities for each class.
- Total Parameters: 101,770.

Simple CNN Architecture:

- Layers:
 - Conv2D: A convolutional layer with 32 filters of size 3x3 and ReLU activation. This layer is responsible for learning spatial features from the input image.
 - MaxPooling2D: A pooling layer with a 2x2 window that reduces the spatial dimensions of the feature maps, making the model more efficient and robust.
 - Flatten: This layer converted the 2D feature maps from the previous layer into a 1D vector.
 - Dense: A fully connected layer with 128 neurons and ReLU activation, used for high-level classification.
 - Dense (Output): The final output layer with 10 neurons using the Softmax activation function.
- Total Parameters: 693,962

2. Training Process and Use of Callbacks

➤ Data Preprocessing:

- The MNIST image data was loaded, and pixel values were normalized from the [0, 255] range to [0, 1] by dividing by 255.0. This helps the model train more effectively.
- For the CNN model, the input images were reshaped from (60000, 28, 28) to (60000, 28, 28, 1) to add the required channel dimension for convolutional layers.

➤ Training Parameters:

- Optimizer: adam
- Loss Function: sparse_categorical_crossentropy
- Batch Size: 32
- Epochs: A maximum of 10.

➤ Exploration of Callbacks: Two key Keras callbacks were used to manage and optimize the training process:

- i. EarlyStopping: This callback monitored the validation loss (val_loss). Training was configured to halt automatically if the val_loss did not improve for 3 consecutive epochs. This was crucial in preventing the models from overfitting.
- ii. ModelCheckpoint: This callback monitored the validation accuracy (val_accuracy) and saved the model weights only when the accuracy improved. This ensured that regardless of when training stopped, we retained the version of the model that performed best on the validation data.

➤ Challenges Faced:

The primary challenge encountered during training was overfitting. As seen in the training logs for both models, after a few epochs, the validation loss began to increase while the training accuracy continued to improve. This indicates the model was starting to memorize the training data rather than generalizing. The EarlyStopping callback was the solution to this challenge, as it effectively stopped the training process before the model could overfit significantly.

3. Performance Evaluation and Comparison

The final performance was measured by evaluating the best saved models on the unseen test dataset.

ANN Final Test Accuracy: 97.74%

CNN Final Test Accuracy: 98.64%

Comparison of Results (ANN vs. CNN):

The results clearly demonstrate the superiority of the CNN for this image classification task. The CNN achieved a higher test accuracy (98.64%) compared to the ANN (97.74%). This is because the CNN's architecture, with its convolutional and pooling layers, is specifically designed to understand the spatial hierarchy of features in an image. It learns to recognize patterns like edges and shapes regardless of their position. The ANN, by flattening the image into a vector, loses this critical spatial information and is therefore less effective.

2. **Build a Simple ANN and CNN for a Binary Classification Task:**

1. Architectures of ANN & CNN

To address the binary classification task of distinguishing between cats and dogs, a simple Artificial Neural Network (ANN) and a Convolutional Neural Network (CNN) were created.

Simple ANN Architecture:

This model was designed to process the images using only standard fully connected layers.

➤ Layers:

- Rescaling: The input layer normalized the raw pixel values from the [0, 255] range to [0, 1]. It was configured for an input shape of (150, 150, 3).
- Flatten: This layer converted the 3D image arrays (150x150x3) into a massive 1D vector containing 67,500 features.
- Dense: A fully connected hidden layer with 128 neurons using the ReLU activation function.
- Dense (Output): The final layer with a single 1 neuron and a Sigmoid activation function to output a probability between 0 (cat) and 1 (dog).

- Parameters: As shown in the model summary, this architecture resulted in a staggering 8,640,257 trainable parameters. The vast majority of these were created by the connection between the 67,500 flattened inputs and the 128 neurons in the first dense layer.

Simple CNN Architecture:

This model was designed to leverage convolutional layers to intelligently extract features from the images.

- Layers:
 - Rescaling: An input layer to normalize pixel values for an input shape of (150, 150, 3).
 - Conv2D: A convolutional layer with 32 filters (3x3) and ReLU activation to learn basic spatial features.
 - MaxPooling2D: A pooling layer to down-sample the feature maps and generalize the learned features.
 - Conv2D: A second convolutional layer with 64 filters (3x3) and ReLU activation to learn more complex, higher-level features.
 - MaxPooling2D: A second pooling layer to further reduce dimensionality.
 - Flatten: This layer converted the final 2D feature maps from the convolutional blocks into a 1D vector.
 - Dense: A fully connected layer with 128 neurons and ReLU activation for classification based on the extracted features.
 - Dense (Output): The final output layer with 1 neuron and a Sigmoid activation.
- Parameters: This architecture contained 10,636,481 trainable parameters. Interestingly, while the convolutional layers themselves were parameter-efficient, the flattening of the final feature map resulted in a very large dense layer, leading to a high total parameter count.

2. Training Process and Use of Callbacks

- Data Preprocessing and Training Parameters:
 - A standard, pre-filtered version of the Cats vs. Dogs dataset was used for practical reasons, containing 2,000 training images and 1,000 validation/test images.
 - The `image_dataset_from_directory` utility was used to load the data. This function automatically handled resizing all images to 150x150 pixels and assigning labels based on their parent folder ('cats' or 'dogs').

- Models were compiled with the adam optimizer and binary_crossentropy loss function, which is standard for two-class problems. Training was run with a batch_size of 32 for a maximum of 10 epochs.
- Exploration of Callbacks:
 - EarlyStopping: This callback was crucial. It monitored the validation loss (val_loss) and was configured with a patience of 3. In both experiments, it successfully halted the training process once the model's performance on the validation set stopped improving, which saved significant time and prevented the models from overfitting.
 - ModelCheckpoint: This callback monitored the validation accuracy (val_accuracy) and ensured that only the best version of each model was saved to a file (.keras). This allowed us to evaluate the peak performance of each model, regardless of when the training was stopped.
- Challenges Faced:
 - The primary challenge was the inherent difficulty of the dataset. Unlike MNIST, these images have varied backgrounds, lighting, and object positions.
 - For the ANN, this complexity was an insurmountable obstacle. The model's architecture, with its 8.6 million parameters in a single layer, was completely overwhelmed. It was unable to learn meaningful patterns from the high-dimensional, unstructured input vector, as shown by its unstable training and near-random performance.
 - For the CNN, the main challenge was the computational cost. The deeper architecture and larger image sizes meant each training epoch took considerably longer than on the MNIST dataset.

3. Performance Evaluation and Comparison

The best-performing version of each model was loaded and evaluated on the unseen test dataset.

ANN Final Test Accuracy: 57.30%

CNN Final Test Accuracy: 69.00%

Comparison and Analysis: The results provide a dramatic and conclusive illustration of the power of Convolutional Neural Networks.

The ANN performed only slightly better than a random guess (50%), confirming its inability to handle complex, real-world image data. By flattening the image, it discards all spatial information and is forced to learn from an enormous, unstructured vector, a task for which it is fundamentally ill-suited.

The CNN, in stark contrast, demonstrated a clear ability to learn. By using its convolutional layers to identify and build upon spatial features in a hierarchical manner, it was able to find patterns that distinguished cats from dogs, achieving a significantly higher accuracy. This proves that for any serious computer vision task, an architecture that respects the spatial nature of images, like a CNN, is not just an option, it is a necessity.